

mai. 18, 2026

Testes de unidade e integração

- parte 2

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com

Anexos  Testes de unidade e integração - parte 2

Resumo

O encontro detalhou a implementação de testes de integração robustos via contêineres e arquitetura Vertical Slice.

Arquitetura e Testes Integrados

Eventos de domínio devem ser imutáveis e sem lógica de negócio. A equipe validou o uso de contêineres Docker via Testcontainers para garantir testes de integração determinísticos.

Estratégias de Mocking e Infra

Dependências externas foram isoladas utilizando mocks para evitar falsos positivos. A configuração de infraestrutura de teste priorizou bancos de dados reais em vez de versões em memória.

Pipeline e Qualidade Técnica

A implementação de agentes de build e práticas de CI/CD foi discutida para ambientes reais. A decisão principal foi adotar a arquitetura Vertical Slice para otimizar a testabilidade.

O que achou deste resumo? [Sim](#) ou [Não é útil?](#)

Próximas etapas

- ☐ [Bruno Joaquim] Link do GitHub: Disponibilizar a URL contendo o código da aplicação de exemplo utilizada para os testes de integração.
- ☐ [O grupo] Praticar RabbitMQ: Baixar o repositório do projeto e tentar configurar o Rabbit MQ em um container no lugar do N substitute para fins de prática.

- ☐ [Márcio das Chagas Ramos] Pesquisar Setup XUnit: Pesquisar o atributo do XUnit que permite a execucao de metodos de configuracao global antes do inicio da bateria de testes.
- ☐ [O grupo] Validar RabbitMQ: Implementar teste de integração usando um contêiner para o RabbitMQ. Realizar a publicação e o consumo de eventos para validar a interação corretamente.
- ☐ [Bruno Joaquim] Compartilhar artigos auditoria: Enviar materiais e referências sobre auditoria e histórico de alterações em dados para Márcio.
- ☐ [Bruno Joaquim] Incluir arquivo seed: Adicionar o arquivo de seed ao repositório de testes de integracao. Este arquivo e necessario para assegurar o funcionamento dos testes.

Detalhes

- **Eventos de Domínio e Serviços Externos:** Farley Souza questionou se faz sentido chamar um serviço externo dentro de um evento de domínio ([00:00:00](#)). Bruno Joaquim esclareceu que os eventos de domínio devem ser imutáveis, funcionando como um contrato que comunica algo que aconteceu, sem conter lógica de comportamento ou chamadas de serviço, visto que eles representam dados e não deveriam realizar processamento adicional ([00:01:33](#)).
- **Uso de Canais no .NET para Monólitos:** Márcio das Chagas Ramos perguntou se é apropriado usar eventos de domínio em aplicações monolíticas. Bruno Joaquim sugeriu que, nesses casos, o uso de "Canais" (Channels) do .NET é uma alternativa eficaz, permitindo aplicar o conceito de mensageria de forma assíncrona dentro do próprio processo, sem a necessidade de infraestrutura externa ([00:02:57](#)).
- **Características e Limitações dos Canais:** Bruno Joaquim explicou que os Canais são filas em memória que desacoplam o produtor do consumidor ([00:03:57](#)). Embora úteis para tarefas assíncronas como envio de e-mails ou implementação do padrão Outbox sem atrasos de *polling*, eles não possuem persistência; se o serviço reiniciar, as mensagens em memória são descartadas, funcionando sob a lógica de "processamento de no máximo uma vez" ([00:05:00](#)) ([00:08:24](#)).
- **Comunicação entre Contextos Delimitados:** Farley Souza indagou se o uso de produtor-consumidor em memória seria viável para comunicação entre contextos delimitados (*bounded contexts*). Bruno Joaquim explicou que é

tecnicamente possível, mas ressaltou o risco de perda de dados e esclareceu que os Canais operam em uma relação um-para-um, diferenciando-se do modelo de assinatura do *Mediator* ([00:09:26](#)).

- **Introdução aos Testes de Integração:** Bruno Joaquim iniciou a implementação prática de testes de integração, enfatizando o foco no "caminho feliz" dos casos de uso mais complexos ([00:11:50](#)). O objetivo é validar o funcionamento da aplicação como um todo, evitando acoplar os testes a detalhes de implementação, como o uso de *controllers* ou *Mediator*, para reduzir a ocorrência de falsos positivos ([00:13:07](#)).
- **Mapeamento de Dependências do Caso de Uso:** O caso de uso identificado como mais complexo é a criação de uma transação bancária ([00:14:35](#)). As dependências mapeadas incluem um banco de dados SQL Server para persistência, uma API externa para conversão de moeda e o RabbitMQ para despacho de eventos de domínio, como o *Budget Alert Domain Event* ([00:15:52](#)).
- **Categorização de Dependências:** Bruno Joaquim categorizou o SQL Server como uma dependência gerenciada, a ser executada via contêiner real, enquanto a API externa e o RabbitMQ foram classificados como não gerenciados, recomendando o uso de *Mocks* para isolá-los ([00:19:56](#)).
- **Configuração da Infraestrutura de Teste:** A biblioteca `Microsoft.AspNetCore.Mvc.Testing` foi introduzida para permitir a criação de uma instância da API em memória através da classe `WebApplicationFactory` ([00:21:25](#)). Essa estrutura possibilita a manipulação do contêiner de Injeção de Dependência (IOC) para substituir registros de serviços reais por implementações de teste ([00:24:23](#)).
- **Simulação de Autenticação e Carga Inicial de Dados:** Bruno Joaquim decidiu substituir o *middleware* de autenticação por uma implementação *fake* para simplificar o teste, eliminando a necessidade de validação de tokens ([00:25:52](#)). Ele enfatizou a importância de realizar um *seed* no banco de dados com usuário, conta e categoria antes da execução do teste, garantindo que o cenário esteja montado ([00:28:39](#)).
- **Uso de Contêineres Docker com Testcontainers:** Bruno Joaquim demonstrou o uso da biblioteca *Testcontainers* para gerenciar o SQL Server, o que automatiza a subida de um contêiner real, randomiza portas para evitar conflitos e garante um ambiente de teste isolado ([00:30:55](#)) ([00:33:49](#)).
- **Vantagens de Bancos Reais versus Em Memória:** Durante a discussão, Bruno Joaquim desencorajou o uso de bancos de dados em memória para testes de integração ([00:35:15](#)). Ele explicou que bancos reais, rodando em contêineres,

evitam falsos positivos e garantem que comportamentos específicos da versão do SQL Server (como sintaxes de consulta) sejam corretamente validados ([00:36:23](#)).

- **Ciclo de Vida do Teste:** A interface `ISyncLifetime` foi utilizada para gerenciar o ciclo de vida do contêiner, utilizando os métodos `InitializeAsync` para subir o SQL Server e `DisposeAsync` para interrompê-lo ao final da execução ([00:38:56](#)).
- **População do Banco de Dados com Scripts:** Bruno Joaquim explicou o uso de um arquivo `Seed.sql` para garantir que as tabelas contenham as entidades necessárias antes do teste iniciar, executando o script diretamente no contêiner via `container.Exec` ([00:40:24](#)).
- **Gerenciamento de Connection Strings:** Como os contêineres SQL são recriados, a *connection string* deve ser dinâmica. Bruno Joaquim mostrou que é possível sobrescrever as configurações do `appsettings.json` definindo variáveis de ambiente, que possuem precedência na leitura das configurações da aplicação ([00:43:11](#)).
- **Mocking do RabbitMQ com NSubstitute:** Para validar a comunicação com o RabbitMQ, Bruno Joaquim utilizou a biblioteca *NSubstitute* para criar *mocks* das interfaces `IChannel` e `IConnection` ([00:48:29](#)). Essa abordagem permite verificar se as publicações ocorrem corretamente sem a necessidade de uma infraestrutura física de mensageria durante o teste ([00:47:11](#)) ([00:50:01](#)).
- **Considerações sobre Pipelines de CI/CD:** Rafael Mendonça questionou sobre a execução desses testes em ambientes de *deploy* ([00:53:28](#)). Bruno Joaquim esclareceu que o agente de *build* da *pipeline* precisa apenas ter o Docker instalado para executar os contêineres da mesma forma que ocorre na máquina de desenvolvimento local ([00:54:44](#)).
- **Execução de Migrações de Banco de Dados:** Márcio das Chagas Ramos perguntou sobre a criação das tabelas, e Bruno Joaquim explicou que as migrações do Entity Framework são aplicadas durante a inicialização da aplicação. Ele mencionou que, para cenários mais complexos de produção, é possível utilizar o `dotnet ef bundle` para um controle maior das migrações na *pipeline* ([00:55:53](#)).
- **Execução de Script de Banco de Dados:** Bruno Joaquim explica a etapa de execução do CID (banco de dados) para garantir que os dados de teste estejam persistidos. A estratégia envolve usar as mesmas informações do CID para manter a consistência, sendo recomendado rodar testes de forma

sequencial com limpeza pós-execução para evitar conflitos de dados ([00:58:39](#)).

- **Abordagem para Testes de Múltiplos Relatórios:** Márcio das Chagas Ramos questiona a melhor estratégia para validar múltiplos relatórios após uma única inserção de dados ([01:00:19](#)). Bruno Joaquim orienta que criar três testes separados é a abordagem preferida, pois evita testes muito extensos e, como novos dados são gerados, reduz a probabilidade de interferência entre os resultados ([01:01:38](#)).
- **Otimização da Configuração de Testes:** Bruno Joaquim discute a possibilidade de implementar métodos de setup globais para verificar a existência de dados no banco antes da inserção, evitando duplicidade. É sugerido o uso de atributos específicos do XUnit para executar configurações apenas uma vez antes dos testes, otimizando o desempenho ([01:02:37](#)).
- **Gestão de Migrações em Testes:** Farley Souza questiona o momento de execução das migrações do banco de dados. Bruno Joaquim esclarece que a aplicação executa as migrações automaticamente no momento da inicialização, independente de ser em teste ou produção, e que o teste utiliza uma connection string específica para um contêiner Docker ([01:03:49](#)).
- **Configuração de Requisição DTO:** Bruno Joaquim descreve a criação do DTO de requisição, permitindo reutilizar estruturas de caso de uso existentes ([01:04:44](#)). O teste de integração valida a interação com a API, passando valores como 150 para uma transação e verificando o código de status 201, diferenciando-se de um teste de unidade pela interação real com a API em execução ([01:05:56](#)).
- **Ciclo de Vida de Contêineres em Testes:** Bruno Joaquim demonstra o uso do Docker Desktop para gerenciar contêineres, explicando que o contêiner de teste tem o tempo de vida limitado à execução do teste ([01:07:15](#)). Essa abordagem valida a injeção de dependência, migrações e persistência de dados de forma real ([01:08:31](#)).
- **Inspeção de Dados no Banco:** Utilizando o DBeaver, Bruno Joaquim verifica a persistência dos dados no SQL Server dentro do contêiner, confirmando que os dados foram inseridos corretamente após a requisição da API. As portas dos contêineres são randomizadas para evitar conflitos ([01:09:45](#)).
- **Mitigação de Dependências Externas:** Bruno Joaquim enfatiza o risco de utilizar APIs reais em testes, o que pode gerar falsos positivos caso o serviço externo falhe. A recomendação é mapear dependências, como Rabbit MQ ou APIs de terceiros, mocando-as para manter a aplicação sob controle total durante o teste ([01:11:05](#)).

- **Validação de Comportamento Observável:** Bruno Joaquim argumenta que apenas validar o status code não é suficiente para um teste robusto ([01:13:28](#)). É necessário validar o resultado, como a inserção real no banco de dados, para garantir que o comportamento esperado da aplicação foi cumprido ([01:14:30](#)).
- **Acesso a Serviços via Escopo:** Ao interagir com o DB Context em testes, Bruno Joaquim explica a necessidade de criar um escopo para simular a requisição HTTP, garantindo que o ciclo de vida (scoped) dos serviços seja respeitado, em vez de tentar acessar o serviço diretamente ([01:15:45](#)).
- **Asserções e Determinismo:** Bruno Joaquim detalha a validação de dados persistidos, como valor e moeda, destacando a importância de manter testes determinísticos ([01:18:27](#)). A dependência de dados voláteis (como cotação de dólar) torna o teste não determinístico, sugerindo a necessidade de mockar APIs externas ([01:19:49](#)).
- **Uso de Ferramentas para Mocking:** Para garantir resultados determinísticos, Bruno Joaquim explora o uso de ferramentas como Flurl ou Mocaco para simular comportamentos de APIs externas. Isso permite validar se a aplicação realiza requisições corretamente sem depender da disponibilidade de serviços de terceiros ([01:23:07](#)) ([01:25:37](#)).
- **Configuração do Contêiner Mockaco:** Bruno Joaquim apresenta o Mocaco como uma ferramenta para subir uma API local em um contêiner, permitindo pré-configurar respostas para rotas específicas ([01:25:37](#)). Esta solução é eficaz para evitar que testes de integração falhem devido a instabilidades em serviços externos ([01:26:52](#)).
- **Testes de Contrato e Versão:** Igor Romulo questiona como lidar com mudanças inesperadas em APIs de terceiros. Bruno Joaquim responde que, embora testes de integração não resolvam isso, a prática recomendada é trabalhar com versionamento de API (V1, V2) para evitar quebras de contrato (breaking changes) ([01:28:11](#)).
- **Testes com Múltiplos Contêineres:** Bruno Joaquim demonstra a subida de dois contêineres (banco de dados e Mockaco) para criar um ambiente de teste completo. A atualização da connection string no app settings garante que a aplicação aponte para os contêineres corretos, possibilitando uma validação determinística da lógica de conversão ([01:30:41](#)).
- **Validação de Mensageria:** Bruno Joaquim reforça a importância de garantir que métodos de publicação, como o envio para filas do Rabbit MQ, sejam chamados corretamente com os parâmetros esperados, mesmo utilizando

mocks ([01:33:59](#)). Isso assegura que a integração com o Rabbit MQ funcione como pretendido ([01:35:26](#)).

- **Foco na Resposta do Caso de Uso:** Bruno Joaquim reforça que testes de integração devem focar no resultado observável do caso de uso (criação de registros, retornos de status) em vez de detalhes de implementação técnica. A abordagem de utilizar contêineres proporciona controle total, evitando resultados falsos positivos ([01:36:46](#)).
- **Design de Código e Testabilidade:** Farley Souza e Bruno Joaquim discutem como a facilidade ou dificuldade de escrever testes é um forte indicativo do design do código. Um design bem estruturado gera, como consequência, um código testável e amigável, enquanto a necessidade de mocks excessivos pode sinalizar um design incorreto ([01:39:30](#)).
- **Arquitetura Vertical Slice:** Bruno Joaquim destaca que a arquitetura Vertical Slice facilita a criação de casos de uso específicos, evitando controladores inchados e dependências desnecessárias. Isso torna o código mais eficiente e o teste de integração mais direto, eliminando a necessidade de testes de unidade excessivamente complexos e mocados ([01:42:10](#)) ([01:45:26](#)).
- **Tratamento Híbrido de Erros:** Bruno Joaquim propõe uma abordagem híbrida onde exceções são usadas para erros de programação (bugs da aplicação) e padrões de resultados (Result Pattern) são utilizados para validações de input do usuário ([01:46:30](#)). Essa estratégia separa claramente erros de lógica interna de erros de validação de dados ([01:47:38](#)).
- **Referências e Material de Estudo:** São disponibilizados links para repositórios com exemplos de implementação em Vertical Slice e com o uso de controladores ([01:48:46](#)). Bruno Joaquim enfatiza que a compreensão do código se torna mais fácil através de exemplos práticos desenvolvidos passo a passo, preferindo estes a projetos de terceiros ([01:50:06](#)).
- **Feedback sobre Prática vs Teoria:** Márcio das Chagas Ramos e Bruno Joaquim conversam sobre a preferência por aulas práticas em vez de apresentações de slides ([01:52:11](#)). Bruno Joaquim recomenda um livro específico sobre Domain Driven Design para aprofundamento, ressaltando a importância de aplicar conhecimentos em cenários reais para fixação do aprendizado ([01:53:20](#)).
- **Qualidade Além da Cobertura de Testes:** A discussão encerra enfatizando que métricas de cobertura de testes não devem ser o único indicador de qualidade ([01:54:19](#)). É crucial verificar se os testes protegem contra regressões e se não estão excessivamente acoplados a detalhes de

implementação, superando a prática comum de apenas buscar alta cobertura com testes unitários mocados ([01:55:29](#)).

- **Abordagem de Ensino e Auditoria de Sistemas:** Bruno Joaquim destaca que o ensino ministrado é pautado em experiências reais, enquanto Márcio das Chagas Ramos solicita sugestões de métodos para implementar funcionalidades de auditoria em sistemas, focando na necessidade de manter um histórico de movimentações e alterações de cadastros ([01:57:31](#)).
- **Configuração de Pipeline no Azure DevOps:** Bruno Joaquim demonstra como configurar um agente de build auto-hospedado (self-hosted) utilizando uma máquina local, explicando a necessidade de criar um pool de agentes e configurar um token de acesso pessoal para permitir que a infraestrutura local execute jobs agendados no DevOps ([01:58:28](#)).
- **Pré-requisitos de Execução da Pipeline:** Foi discutido que, para a execução correta da pipeline, é necessário ter o .NET SDK, o runtime e o Docker instalados na máquina agente, com Bruno Joaquim confirmando que é possível automatizar a instalação dessas dependências diretamente através dos passos da própria pipeline ([02:01:45](#)).
- **Gestão de Custos em Serviços de Nuvem:** Farley Souza expressa preocupação com possíveis cobranças elevadas ao vincular cartões de crédito em plataformas de nuvem, levando Márcio das Chagas Ramos a recomendar a configuração de limites de gastos e a utilização dos níveis gratuitos, mencionando a disponibilidade de 1800 minutos gratuitos para pipelines no Azure ([02:03:05](#)).
- **Testes de Integração em Filas Assíncronas:** Jonathan Jimenez questiona a metodologia ideal para testar consumidores de API que utilizam filas assíncronas como RabbitMQ ou SQS; Bruno Joaquim valida a abordagem de publicar um evento, aguardar o processamento através de uma verificação de estado no banco de dados (polling) e, em seguida, realizar as asserções necessárias ([02:04:42](#)).
- **Consistência do Conteúdo do Curso:** Farley Souza questiona se o conteúdo ministrado varia entre diferentes turmas, ao que Bruno Joaquim esclarece que a grade curricular é mantida consistente, embora a interação dos alunos e os casos de uso compartilhados possam variar a dinâmica das aulas ([02:07:00](#)).
- **Compartilhamento de Conteúdo Adicional:** Bruno Joaquim menciona a existência de um canal no YouTube onde disponibiliza materiais complementares sobre Docker, testes de integração, Dapr e integrações com Apache Kafka, visando auxiliar os participantes além das aulas ([02:08:01](#)).

- **Gerenciamento de Bancos de Dados para Testes:** Jorge Oliveira indaga sobre a separação de ambientes de banco de dados para desenvolvimento e testes; Bruno Joaquim recomenda não compartilhar bancos de dados entre a equipe de desenvolvimento para evitar resultados de testes imprevisíveis (falsos positivos) e sugere versionar o seed de dados junto ao código-fonte ([02:09:13](#)).
- **Estratégias de Seeding em Testes:** Wallace Silva discute como realizar o seed de banco de dados em múltiplos casos de uso, e Bruno Joaquim explica que o ideal é centralizar o seed ou utilizá-lo dentro de inicializadores de classe para evitar erros de duplicidade de chaves, garantindo que o comportamento seja previsível ([02:10:13](#)).
- **Desempenho de Contêineres em Testes:** Jonathan Jimenez levanta preocupações sobre a performance da máquina ao rodar múltiplos contêineres simultâneos; Bruno Joaquim assegura que contêineres são leves e não consomem muitos recursos, embora sugira otimizar a arquitetura para evitar instâncias desnecessárias da aplicação ([02:12:47](#)).
- **Encerramento da Reunião:** Bruno Joaquim finaliza a sessão anunciando a próxima masterclass que será conduzida por Gabriel na quarta-feira seguinte ([02:14:03](#)).

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.